

Software Design and Architecture (COE692) Lab 5

Duration 2 weeks

Group work and Group submission¹

Objectives

The main objectives of this lab are as follows:

- Implementing Asynchronous messaging between microservices using Kubernetes Queues
- Implementing Synchronous messaging between microservice in a cloud environment
- Pull the microservices of your project as docker containers in docker hub
- Set up communications between containers using Kubernetes
- Deploy all microservice on the Google Cloud
- Expose the final project on a public api, such that everyone can test and see the work!

Problem description

The first part of the lab asks you to complete asynchronous messaging between microservices based on kubemq technology. The second part of the lab asks you to complete synchronise communication between microservices, and between each microservice and its own database (Because a database will be deployed in a container). The third part of the lab asks you to create docker containers for all microservices and their databases and push them to Docker Hub. The fourth part of the lab asks you to deploy your project using Kubernetes pods and services over Google Cloud.

Requirements

- Installing all required tools based on the document “Pre-requirements of lab 5”.
- A very comprehensive review of Week 10 for theories and Week 11 for technical aspects of the project.

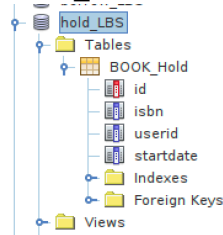
Part 1: Implementing Asynchronous messaging

In this part, you must recognize micro-services that need asynchronous messaging, their databases and

¹ All files needed to be deployed on a Google Cloud Account. Just one Google Cloud account is enough for each team. If two team-members are in different sections (should be handled by the same TA), the group deadline is the latest deadline in the team (e.g., in a group of two members with deadlines of Tuesday and Thursday, the group deadline will be on Thursday).

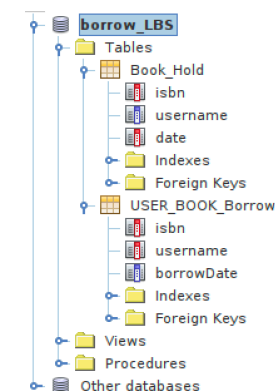
affected tables and then implement publishing a message and receiving messages. In the following I will comprehensively describe how I implemented this technique in my own Library Management System (LMS) project. You must complete this part for your own project².

- **Dependent microservices description:** In LMS project, there are two microservices hold_book and borrow_book that need to communicate asynchronously. Borrow_book can borrow a book to a user only if it is not on hold by another user. Hence borrow_book needs to know which books are on hold by whom. Whenever a new hold created by the hold_book microservice, a new message must be published to interested consumers. The borrow_book microservice is subscribed to the Hold Book Channel, when it can see and act upon receiving new messages.
- **Database Description:** hold_book microservice has the hold_LBS database. This database has the BOOK_Hold table, which has id, isbn, userid, startdate columns (see the following schema):



When a request for holding a book is performed, a new row will be added to the BOOK_Hold table. Also a message will be published in a channel named “hold_book_channel” with the following format:

HOLD:\$BOOKISBN:\$USERNAME:\$ExpireDate, when \$BOOKISBN is the isbn of the book is on hold, :\$USERNAME is the username who hold a book, :\$ExpireDate is when hold is no longer valid in this format “yyyy-MM-dd”. The borrow_book microservice has the borrow_LBS database with following tables:



USER_BOOK_Borrow is a table related to borrowing books. BOOK_Hold is table that keeps information about books that are on hold received from hold_book micro service.

² Please note that your project may need to have multiple asynchronous messaging to perform well. But for the purpose of this lab, describing and implementing just one is enough.

Coding: In order to implement messaging, I added a new class to the HoldBook project, business package. This class is named Messaging and has the following structure:

```
1 package ryerson.ca.holdbook.business;
2
3 import io.kubemq.sdk.basic.ServerAddressNotSuppliedException;
4 import io.kubemq.sdk.event.Event;
5
6
7
8
9
10 import io.kubemq.sdk.tools.Converter;
11
12 import javax.net.ssl.SSLException;
13 import java.io.IOException;
14
15
16 public class Messaging {
17
18
19
20     public static void sendmessage(String message) throws IOException{
21         String channelName = "hold_book_channel",
22             clientID = "hold-book-subscriber";
23         String kubeMQAddress = System.getenv("kubeMQAddress");
24
25         io.kubemq.sdk.event.Channel channel = new io.kubemq.sdk.event.Channel(channelName, clientID, false,
26             kubeMQAddress);
27
28         channel.setStore(true);
29         Event event = new Event();
30         event.setBody(Converter.ToByteArray(message));
31         event.setEventId("event-Store-");
32         try {
33             channel.SendEvent(event);
34         } catch (SSLException e) {
35             System.out.printf("SSLException: %s", e.getMessage());
36             e.printStackTrace();
37         } catch (ServerAddressNotSuppliedException e) {
38             System.out.printf("ServerAddressNotSuppliedException: %s", e.getMessage());
39             e.printStackTrace();
40         }
41     }
42 }
43
44
```

Figure 1

Messaging class has a sendmessage method that sends a message to the “hold_book_channel”. ClientID could be anything here, I chose “hold-booksubscriber”. The important part is line #23, when the address for sending this message must be assigned. I do not know the exact address on the cluster, so I assume there will be an environment variable which will be set at run time. This environment variable is named “kubeMQAddress”. Lines #25, and #26 creates a channel and set it to be stored (Please refer to the lecture to find out more about different channel types). In line #30, message will be converted to an appropriate format. Finally line #33 sends the message.

You can use the exact same method in your codes by some customizations (Example: changing the channel name).³

³ All codes of this project will be available on your D2L in a zip file named Lab5codes.zip

In the BorrowBook project, in the

- **Sending message Code:**

In HOLDBOOK project, in the HoldBusiness class, I changed the hold method to send the appropriate message using the Messaging class as follow:

```
1  package ryerson.ca.holdbook.business;
2
3
4  import io.kubemq.sdk.basic.ServerAddressNotSuppliedException;
5  import java.io.IOException;
6  import java.sql.SQLException;
7  import java.time.LocalDate;
8  import java.time.format.DateTimeFormatter;
9  import ryerson.ca.holdbook.helper.BookHold;
10 import ryerson.ca.holdbook.persistence.BOOK_Hold_CRUD;
11
12
13 public class HoldBusiness {
14
15
16     public BookHold getBooks(String isbn) {
17         BookHold bs = BOOK_Hold_CRUD.getHoldBook(isbn);
18
19         return (bs);
20     }
21
22     public boolean hold(String isbn, String userid) throws ClassNotFoundException, SQLException,
23         ServerAddressNotSuppliedException, IOException, InterruptedException {
24         boolean success = false;
25
26         success = BOOK_Hold_CRUD.addHold(isbn, userid);
27         if(success){
28             DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
29             LocalDate date = LocalDate.now();
30
31             LocalDate exDate = date.plusDays(14);
32
33             Messaging.sendMessage("HOLD:"+isbn+": "+userid+": "+exDate.format(formatter));
34         }
35
36
37         return success;
38     }
39
40
41 }
42
43
```

Figure 2

In the BorrowBook project, I created a Messaging class in the business package as follows:

```

1 public class Messaging {
2     public static void Receiving_Events_Store(String cname) throws SSLException, ServerAddressNotSuppliedException {
3         String ChannelName = cname, ClientID = "hello-world-subscribe1r";
4         String kubeMQAddress = System.getenv("kubeMQAddress");
5         Subscriber subscriber = new Subscriber(kubeMQAddress);
6         SubscribeRequest subscribeRequest = new SubscribeRequest();
7         subscribeRequest.setChannel(ChannelName);
8         subscribeRequest.setClientID(ClientID);
9         subscribeRequest.setSubscribeType(SubscribeType.EventsStore);
10        subscribeRequest.setEventsStoreType(EventsStoreType.StartAtSequence);
11        subscribeRequest.setEventsStoreTypeValue(1);
12
13        StreamObserver<EventReceive> streamObserver = new StreamObserver<EventReceive>() {
14            @Override
15            public void onNext(EventReceive value) {
16                try {
17                    String val=(String) Converter.FromByteArray(value.getBody());
18                    System.out.printf("Event Received: EventID: %s, Channel: %s, Metadata: %s, Body: %s",
19                        value.getEventId(), value.getChannel(), value.getMetadata(),
20                        Converter.FromByteArray(value.getBody()));
21                    String[] msgParts = val.split(":");
22                    if(msgParts.length==4){
23                        if(msgParts[0].equals("HOLD")){
24
25                            String isbn=msgParts[1];
26                            String username=msgParts[2];
27                            String date=msgParts[3];
28                            BOOK_Hold_CRUD.addHold(isbn, username, date);
29                        }
30                    }
31                } catch (ClassNotFoundException e) {
32                    System.out.printf("ClassNotFoundException: %s", e.getMessage());
33                    e.printStackTrace();
34                } catch (IOException e) {
35                    System.out.printf("IOException: %s", e.getMessage());
36                    e.printStackTrace();
37                } catch (SQLException ex) {
38                    Logger.getLogger(MyAppServletContextListener.class.getName()).log(Level.SEVERE, null, ex);
39                }
40            }
41
42            @Override
43            public void onError(Throwable t) {
44                System.out.printf("onError: %s", t.getMessage());
45            }
46
47            @Override
48            public void onCompleted() {
49            }
50        };
51
52        subscriber.SubscribeToEvents(subscribeRequest, streamObserver);
53    }
54 }
55 }
56 }
57

```

Figure 3

You must modify lines #16 to #39 for implementing your own logic (Please refer to lecture Week 11 for more details.). Finally Figure 4 is where the receiving messages will be run (See lecturer notes for more details)

```

1
2 package ryerson.ca.borrowbook.endpoint;
3
4
5 import io.kubemq.sdk.basic.ServerAddressNotSuppliedException;
6 import java.util.logging.Level;
7 import java.util.logging.Logger;
8 import javax.net.ssl.SSLException;
9 import javax.servlet.ServletContextEvent;
10 import javax.servlet.ServletContextListener;
11
12 import ryerson.ca.borrowbook.business.Messaging;
13
14
15 public class MyAppServletContextListener
16     implements ServletContextListener{
17
18     @Override
19     public void contextDestroyed(ServletContextEvent arg0) {
20         System.out.println("ServletContextListener destroyed");
21     }
22
23     @Override
24     public void contextInitialized(ServletContextEvent arg0) {
25         Runnable r = new Runnable() {
26             public void run() {
27
28                 try {
29                     Messaging.Receiving_Events_Store("hold_book_channel");
30                 } catch (SSLException ex) {
31                     Logger.getLogger(MyAppServletContextListener.class.getName()).log(Level.SEVERE, null, ex);
32                 } catch (ServerAddressNotSuppliedException ex) {
33                     Logger.getLogger(MyAppServletContextListener.class.getName()).log(Level.SEVERE, null, ex);
34                 }
35             }
36         };
37
38         new Thread(r).start();
39     }
40 }

```

Figure 4

More explanations:

Two important things you must not forget:

1. Editing web.xml of the microservice that has MyAppServletContextListener and add the following:

```

<listener>
  <listener-class>
    ryerson.ca.borrowbook.endpoint.MyAppServletContextListener
  </listener-class>
</listener>

```

Where the package and the class name must be modified based on your own project.

2. Add the following dependencies to the project that uses kubemq

```

<dependency>
  <groupId>io.kubemq.sdk</groupId>
  <artifactId>kubemq-sdk-Java</artifactId>
  <version>1.0.3</version>
</dependency>

```

Exercise 1

You need to create a lab report document: The lab report should be prepared using a word processor and should be stored as a single PDF file. This PDF document should be named as follows:

TeamMember1Name_ TeamMember2Name_Lab5_exc1.pdf. This is what should be included in your pdf file:

1. A paragraph of **Dependent microservices description** and A paragraph of **Database Description** (5/100)
2. Screen shots of codes related to sending and receiving messages (E.g. Figure 2 and Figure 3) and a paragraph description for each figure (10/100)

Part 2: Implementing Synchronous messaging

In this section, you are supposed to change your code (all microservices wherever you used "localhost:8080" or "localhost:3306"). Remember, there will be no localhost on the cloud environment. The exact address will be specified at run time. Using environment variables is a way to achieve this goal. That is how I changed a sample database connection and a sample REST api connection:

```
public class Book_CRUD {  
    private static Connection getCon() {  
        Connection con = null;  
        try {  
            Class.forName("com.mysql.jdbc.Driver");  
            String connection = System.getenv("DB_URL");  
            // String connection ="localhost:3306";  
            con = DriverManager.getConnection("jdbc:mysql://" + connection  
                + "/search_LBS?allowPublicKeyRetrieval=true&useSSL=false", "root", "student");  
            System.out.println("Connection established.");  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
        return con;  
    }  
}
```

Rest API Example:

```

public static BooksXML getServices(String query, String token) throws IOException {
    Client searchclient = ClientBuilder.newClient();
    String searchService= System.getenv("searchService");
    String holdService= System.getenv("holdService");
    WebTarget searchwebTarget
        = searchclient.target("http://"+searchService+"/SearchBooks/webresources/search");
    InputStream is
        = searchwebTarget.path(query).request(MediaType.APPLICATION_XML).get(InputStream.class);
    String xml = IOUtils.toString(is, "utf-8");
    BooksXML books = bookxmltoObjects(xml);
    if (token != null) {
        Client holdclient = ClientBuilder.newClient();
        WebTarget holdwebTarget
            = holdclient.target("http://"+holdService+"/HoldBook/webresources/hold/isOnHold");
        if(books!=null){
            for (Book book : books.getBooks()) {
                InputStream holddata
                    = holdwebTarget.path(book.getIsbn()).queryParam("token", token).
                        request(MediaType.APPLICATION_XML).get(InputStream.class);
                try{
                    Book a=bookholdxmltoObjects(IOUtils.toString(holddata, "utf-8"));
                    if(a!=null)
                        book.setToBeHold(false);
                    else
                        book.setToBeHold(true);
                }
            }
        }
    }
}

```

Part 3: Creating and publishing docker images

Create a folder for all Docker-files. Create a docker file for each microservice and a docker file for each database for a microservice. For example, I created Docker-Lab5Microservices folder inside it I have the following docker files:

- Dockerfile-borrowdb
- Dockerfile-borrowservice
- Dockerfile-holddb
- Dockerfile-holdservice
- Dockerfile-searchdb
- Dockerfile-searchservice
- Dockerfile-fronendservice

Create dumps of your SQL databases and name them appropriately. Copy all sql dumps in the same folder.

For creating a dump of borrow_LBS I used the following command:

```
mysqldump --databases borrow_LBS -user root --password > borrow.sql
```

Enter student when asked about password.

```
student@Linux64-VM:~/Documents/Docker-Lab5Microservices$ mysqldump --database borrow_LBS --user root --password >borrow.sql
Info: Using unique option prefix 'database' is error-prone and can break in the future.
Please use the full name 'databases' instead.
Enter password:
```

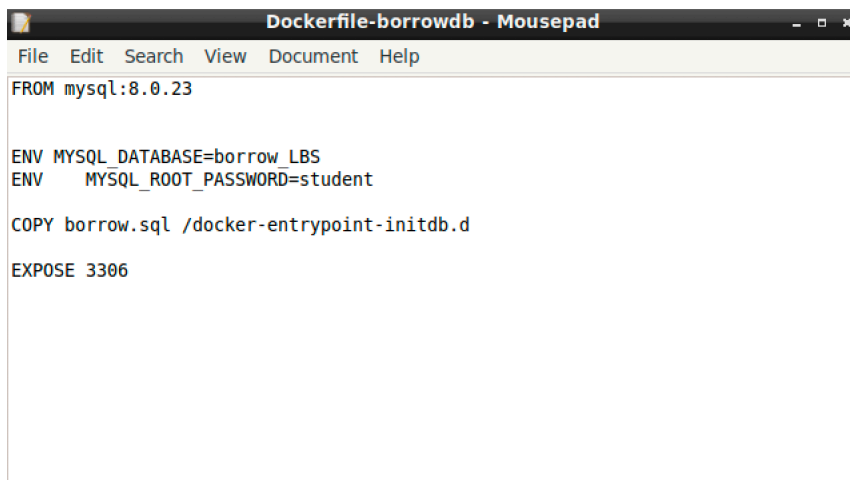
Please note that you must install mysql client with the following command before creating a dump with this method:

```
sudo apt-get install mysql-client
```

Also copy and paste the WAR file of each micro-service project in the same folder. The war file must be found in the following address:

/pathToYourProject/target/ProjectName-1.0-SNAPSHOT.war

The followings are Dockerfile-borrowdb and Dockerfile-borrowdb:

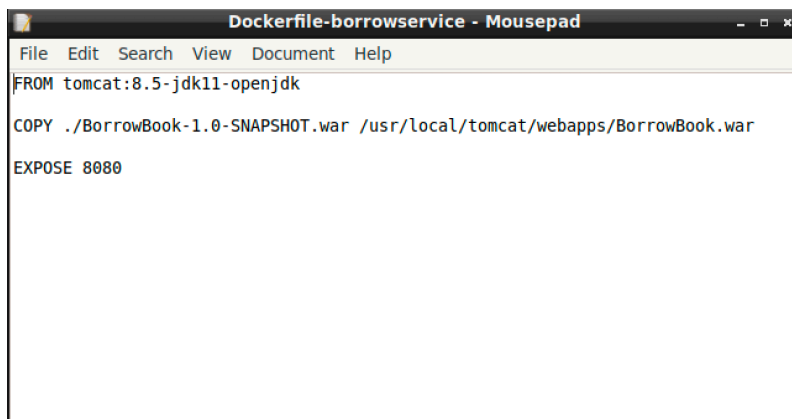
A screenshot of a text editor window titled "Dockerfile-borrowdb - Mousepad". The window has a menu bar with "File", "Edit", "Search", "View", "Document", and "Help". The content of the file is as follows:

```
FROM mysql:8.0.23

ENV MYSQL_DATABASE=borrow_LBS
ENV MYSQL_ROOT_PASSWORD=student

COPY borrow.sql /docker-entrypoint-initdb.d

EXPOSE 3306
```

A screenshot of a text editor window titled "Dockerfile-borrowservice - Mousepad". The window has a menu bar with "File", "Edit", "Search", "View", "Document", and "Help". The content of the file is as follows:

```
FROM tomcat:8.5-jdk11-openjdk

COPY ./BorrowBook-1.0-SNAPSHOT.war /usr/local/tomcat/webapps/BorrowBook.war

EXPOSE 8080
```

Now for creating a docker image follow these instructions:

1. In your terminal cd to the folder of all Docker-files

2. Use `sudo docker build` for creating an image for each micro service. Use `--file` options to point to your desired file inside the folder and use `--tag` option to name it according to Docker hub standard. Here I name my image `fensan/coe692:borrowdb`, where `fensan` is my username in docker hub and `coe692` is my repository. Please review lectures at weeks 9 and 10 for more information on Docker hub.

```
student@Linux64-VM:~/Documents/Docker-Lab5Microservices$ sudo docker build . --file Dockerfile-borrowdb --tag fensan/coe692:borrowdb
sudo: unable to resolve host Linux64-VM: No address associated with hostname
Sending build context to Docker daemon 19.46kB
Step 1/5 : FROM mysql:8.0.23
--> e646c6533b0b
Step 2/5 : ENV MYSQL_DATABASE=borrow_LBS
--> Running in b0c6d8546c79
Removing intermediate container b0c6d8546c79
--> 4e2b47c8d90b
Step 3/5 : ENV MYSQL_ROOT_PASSWORD=student
--> Running in 35750aa6ff4f
Removing intermediate container 35750aa6ff4f
--> 45aae14044d8
Step 4/5 : COPY borrow.sql /docker-entrypoint-initdb.d
--> 2e97894dab86
Step 5/5 : EXPOSE 3306
--> Running in c9853e93cccc
Removing intermediate container c9853e93cccc
--> 4d0db544771c
Successfully built 4d0db544771c
Successfully tagged fensan/coe692:borrowdb
```

3. Use `sudo docker image ls` to see all images created:

```
student@Linux64-VM:~/Documents/Docker-Lab5Microservices$ sudo docker image ls
sudo: unable to resolve host Linux64-VM: No address associated with hostname
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
fensan/coe692        searchdb            79de79401c31       4 seconds ago      546MB
fensan/coe692        holddb              a9d7aee4210f       22 seconds ago     546MB
fensan/coe692        frontendWebApp      3ab5958255e5       40 seconds ago     673MB
fensan/coe692        holdWebApp           e6f4a6b65b3c       About a minute ago 687MB
fensan/coe692        searchWebApp        119760eb7968       About a minute ago 673MB
fensan/coe692        borrowWebApp         a5040d50e3d6       2 minutes ago      687MB
fensan/coe692        borrowdb            4d0db544771c       5 minutes ago      546MB
fensan/coe692        <none>              7ec37df99028       2 days ago         687MB
tomcat               8.5-jdk11-openjdk  6c49a679f4c6       3 days ago         666MB
mysql                8.0.23              e646c6533b0b       5 days ago         546MB
```

Figure 5

4. Important: You can delete unused images with
 - a. `sudo docker image rm "IMAGE ID" -f`
 - b. `sudo docker system prune -a`
 - c. You can find more about docker images commands at:

<https://docs.docker.com/engine/reference/commandline/image/>⁴

⁴ Please note that if you create a lot of unused images, your memory will be full very soon and that makes a lot of problems.

5. Push you images to docker hub:

- First you must login to the docker hub:
`sudo docker login --username=yourhubusername`
- Then push all images with the following command:

```
sudo docker push IMAGENAME
```

```
student@Linux64-VM:~/Documents/Docker-Lab5Microservices$ sudo docker push fensan/coe692
:borrowdb
sudo: unable to resolve host Linux64-VM: No address associated with hostname
The push refers to repository [docker.io/fensan/coe692]
db7b1d12d4e2: Pushed
0f1a72196fff: Mounted from library/mysql
7ea7e9d34a0e: Mounted from library/mysql
31512cb6c95d: Mounted from library/mysql
157806036347: Mounted from library/mysql
c7663e9cd144: Mounted from library/mysql
831892bb8761: Mounted from library/mysql
611b0fca9229: Mounted from library/mysql
ffb2a7b56edd: Mounted from library/mysql
dd7bb34a761f: Mounted from library/mysql
6ff1b592cfd7: Mounted from library/mysql
f4b25930f6fc: Mounted from library/mysql
346fddbbb0ff: Mounted from library/mysql
borrowdb: digest: sha256:a67b0e4e89d53649b6fa0e3180d2c284a49f313352c2404fe2267d43ae64ac
a0 size: 3036
student@Linux64-VM:~/Documents/Docker-Lab5Microservices$
```

6. Your docker hub must have all images:

 fensan / coe692

This repository does not have a description 

 Last pushed: a minute ago

Tags and Scans

VULNERABILITY SCANNING - DISABLED [Enable](#)

This repository contains 10 tag(s).

TAG	OS	PULLED	PUSHED
 frontendWebApp		a minute ago	a minute ago
 holdWebApp		a minute ago	a minute ago
 borrowWebApp		2 minutes ago	2 minutes ago
 searchWebApp		2 minutes ago	2 minutes ago
 searchdb		3 minutes ago	3 minutes ago

Figure 6

Exercise 2

Follow all steps in this part and create images and push them on your docker hub repository. You need to create a lab report document: The lab report should be prepared using a word processor and should be stored as a single PDF file. This PDF document should be named as follows: **TeamMember1Name_TeamMember2Name_Lab5_exc2.pdf**. This is what should be included in your pdf file:

1. A screenshot of all images you created on your system (Like Figure 5 in my project) (5/100)
2. A screenshot of all images you pushed on your docker hub (Like Figure 6 in my project) (5/100)

Part 4: Deploying containers on a Kubernetes cluster

In this part, you need to deploy all containers you created in the last part on a Kubernetes cluster. For this purpose, you need to create a folder for all Kubernetes files (I named my own folder kube). Then create two yamls file:

- A yaml file for deploying all containers (I named it lab5deployment.yaml)
- A yaml file for all services (I named it lab5service.yaml)

Sample yaml files of my project will be shared in the Lab5codes.zip.

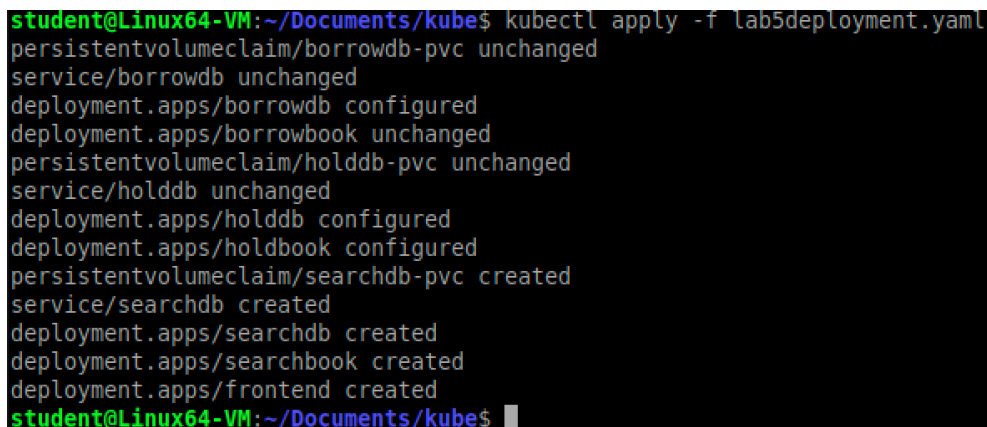
Important points in writing yaml files:

- Do not use tabs
- Use space (at least two spaces) for indent level

Use

```
kubectl apply -f FILENAME
```

To apply your Kubernetes yaml file.



```
student@Linux64-VM:~/Documents/kube$ kubectl apply -f lab5deployment.yaml
persistentvolumeclaim/borrowdb-pvc unchanged
service/borrowdb unchanged
deployment.apps/borrowdb configured
deployment.apps/borrowbook unchanged
persistentvolumeclaim/holddb-pvc unchanged
service/holddb unchanged
deployment.apps/holddb configured
deployment.apps/holdbook configured
persistentvolumeclaim/searchdb-pvc created
service/searchdb created
deployment.apps/searchdb created
deployment.apps/searchbook created
deployment.apps/frontend created
student@Linux64-VM:~/Documents/kube$
```

Figure 7

Use kubectl get pods to get all pods created by Kubernetes on Google Cloud:

```
student@Linux64-VM:~/Documents/kube$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
borrowbook-6598b5d785-2pbnx        1/1     Running   0           3d3h
borrowdb-579fdb4944-rlkvc          1/1     Running   0           2m12s
frontend-79f8df57f6-zvktg          1/1     Running   0           2m8s
holdbook-5b96d7d9bf-h6dxj          1/1     Running   0           2m10s
holddb-5f46464fdf-bjsvq            1/1     Running   0           3d11h
holddb-764fcc7d9b-x9rg4            0/1     ContainerCreating 0           2m10s
searchbook-7894448b8c-lv7xb        1/1     Running   0           2m8s
searchdb-65cc8cd9b9-bqpjq          1/1     Running   0           2m8s
student@Linux64-VM:~/Documents/kube$
```

Figure 8

use

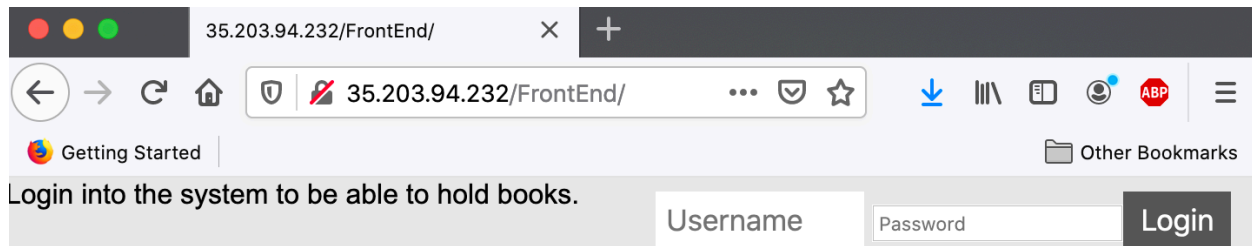
```
kubectl get services my-service
```

To find the ip address of your service:

```
student@Linux64-VM:~/Documents/kube$ kubectl get service frontend
NAME      TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
frontend  LoadBalancer 10.24.13.56  35.203.94.232  80:32325/TCP     5m39s
```

Figure 9

Using external-ip, your app must be reachable from any browser:



Search Books

Search in book titles and book author names

Figure 10

You can delete a pod with the following

kubectl delete pod pod-name

```
student@Linux64-VM:~/Documents/kube$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
borrowbook-6598b5d785-2pbnx        1/1     Running   0           3d3h
borrowdb-579fdb4944-rlkvc          1/1     Running   0           30m
frontend-b6687fbd4-52kbs           1/1     Running   0           87s
holdbook-5b96d7d9bf-h6dxj          1/1     Running   0           30m
holddb-5f46464fdf-bjsvq            1/1     Running   0           3d11h
holddb-764fcc7d9b-x9rg4            0/1     ContainerCreating 0           30m
searchbook-7894448b8c-lv7xb        1/1     Running   0           30m
searchdb-65cc8cd9b9-bqpjq          1/1     Running   0           30m
```

Exercise 3

You need to create a lab report document: The lab report should be prepared using a word processor and should be stored as a single PDF file. This PDF document should be named as follows:

TeamMember1Name_TeamMember2Name_Lab5_exc3.pdf. This is what should be included in your pdf file:

1. A screenshot of all pods you created on your cluster (Like Figure 8 in my project) (5/100)
2. A screenshot of your frontend service information (Like Figure 9 in my project) (5/100)

3. A screenshot of your running website (Like Figure 10 in my project) (5/100)

Deliverables

After finishing the lab, please archive (zip) deliverable files and upload it on D2L submission system with the following format:

< **TeamMember1StudentID_ TeamMember2StudentID** >_lab5.zip

This zip file must include the following file:

- Exercise 1: **TeamMember1Name_ TeamMember2Name_Lab3_exc1.pdf** (15%)
- Exercise 2: **TeamMember1Name_ TeamMember2Name_Lab3_exc2.pdf** (10%)
- Exercise 3: **TeamMember1Name_ TeamMember2Name_Lab3_exc3.pdf** (15%)
- Running Website. You TA is supposed to test your deployed project using the IP address you provided. That is 60% of your final mark.

Good Luck!